Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №3 по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Шапошник А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.11.26

Москва, 2026

Постановка задачи

Вариант 22.
Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2.
Дочерние процессы инвертируют строки.

Общий метод и алгоритм решения

Системные вызовы:

read()	читает до n байт из дескриптора; возвращает число прочитанных, 0 = EOF, -1 = ошибка
write()	пишет до n байт в дескриптор; может записать меньше — нужен цикл
open()	открывает/создаёт файл, возвращает fd
close()	закрывает дескриптор
fork()	создаёт дочерний процесс (копия родителя)
Execv()	заменяет образ процесса программой path; при успехе не возвращается
getpid()	возвращает PID текущего процесса
waitpid()	ждёт завершения дочернего процесса, собирает зомби
ftruncate()	задаёт размер объекта SHM (без него mmap даст SIGBUS)
Mmap()	отображает объект (SHM) в адресное пространство процесса
munmap()	снимает отображение
_exit(s)	немедленное завершение процесса без atexit/сброса stdio (используется в детях после неудачного execv; в вашем коде стоит exit, но _exit корректнее)

Код программы

mmap_client.c

```c
#include <stdbool.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <fcntl.h>
#include <semaphore.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <time.h>

#define BUF_SIZE     4096
#define FILTER_NAME  "reverse"

typedef struct {
    size_t len;
    char   data[BUF_SIZE];
} shm_buffer_t;

static int write_all(int fd, const char *buf, ssize_t len) {
    ssize_t written = 0;
    while (written < len) {
        ssize_t w = write(fd, buf + written, len - written);
        if (w < 0) return -1;
        written += w;
    }
    return 0;
}
```

```
static ssize_t read_line(int fd, char *buf, size_t max) {
    ssize_t total = 0;
    while (total < (ssize_t)max) {
        ssize_t r = read(fd, buf + total, 1);
        if (r < 0) return -1;
        if (r == 0) return total;
        ++total;
        if (buf[total - 1] == '\n') break;
    }
    return total;
}

static bool roll_80(void) {
    int fd = open("/dev/urandom", O_RDONLY);
    if (fd < 0) {
        unsigned seed = (unsigned)time(NULL) ^ (unsigned)getpid();
        return (seed % 100) < 80;
    }
    unsigned char byte;
    do {
        if (read(fd, &byte, 1) != 1) {
            close(fd);
            return true;
        }
    } while (byte >= 200);
    close(fd);
    return (byte % 100) < 80;
}

static int slot_create(const char *shm_name,
                       const char *sem_to_name,
                       const char *sem_from_name,
                       shm_buffer_t **out_buf,
                       int *out_fd,
                       sem_t **out_to,
                       sem_t **out_from)
{
    int fd = shm_open(shm_name, O_RDWR | O_CREAT | O_TRUNC, 0600);
    if (fd == -1) return -1;

    if (ftruncate(fd, sizeof(shm_buffer_t)) == -1) {
        close(fd);
        shm_unlink(shm_name);
        return -1;
    }

    shm_buffer_t *buf = mmap(NULL, sizeof(shm_buffer_t),
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED, fd, 0);
    if (buf == MAP_FAILED) {
        close(fd);
        shm_unlink(shm_name);
        return -1;
    }
    buf->len = 0;

    sem_t *to = sem_open(sem_to_name, O_CREAT | O_EXCL, 0600, 0);
```

```
    if (to == SEM_FAILED) {
        munmap(buf, sizeof(shm_buffer_t));
        close(fd);
        shm_unlink(shm_name);
        return -1;
    }

    sem_t *from = sem_open(sem_from_name, O_CREAT | O_EXCL, 0600, 0);
    if (from == SEM_FAILED) {
        sem_close(to);
        sem_unlink(sem_to_name);
        munmap(buf, sizeof(shm_buffer_t));
        close(fd);
        shm_unlink(shm_name);
        return -1;
    }

    *out_buf  = buf;
    *out_fd   = fd;
    *out_to   = to;
    *out_from = from;
    return 0;
}

static void slot_destroy(shm_buffer_t *buf, int fd,
                         sem_t *to, sem_t *from,
                         const char *shm_name,
                         const char *sem_to_name,
                         const char *sem_from_name)
{
    sem_close(to);
    sem_close(from);
    sem_unlink(sem_to_name);
    sem_unlink(sem_from_name);
    munmap(buf, sizeof(shm_buffer_t));
    shm_unlink(shm_name);
    close(fd);
}

int main(int argc, char **argv) {
    if (argc < 2) {
        const char msg[] = "usage: client path/to/reverse-filter [output_file]\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_SUCCESS);
    }

    const char *filterpath = argv[1];

    int out_fd = STDOUT_FILENO;
    if (argc >= 3) {
        out_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0600);
        if (out_fd == -1) {
            const char msg[] = "error: failed to open output file\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
    }
```

```
char shm1_name[64], shm2_name[64];
char sem_to1[64], sem_to2[64];
char sem_from1[64], sem_from2[64];

snprintf(shm1_name, sizeof(shm1_name), "/lab3_shm1_%d", getpid());
snprintf(shm2_name, sizeof(shm2_name), "/lab3_shm2_%d", getpid());
snprintf(sem_to1,   sizeof(sem_to1),   "/lab3_sem_to1_%d",   getpid());
snprintf(sem_to2,   sizeof(sem_to2),   "/lab3_sem_to2_%d",   getpid());
snprintf(sem_from1, sizeof(sem_from1), "/lab3_sem_from1_%d", getpid());
snprintf(sem_from2, sizeof(sem_from2), "/lab3_sem_from2_%d", getpid());

shm_buffer_t *buf1, *buf2;
int shm1_fd, shm2_fd;
sem_t *to1, *from1, *to2, *from2;

if (slot_create(shm1_name, sem_to1, sem_from1,
                &buf1, &shm1_fd, &to1, &from1) == -1) {
    const char msg[] = "error: failed to create slot 1\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
if (slot_create(shm2_name, sem_to2, sem_from2,
                &buf2, &shm2_fd, &to2, &from2) == -1) {
    slot_destroy(buf1, shm1_fd, to1, from1,
                 shm1_name, sem_to1, sem_from1);
    const char msg[] = "error: failed to create slot 2\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}

pid_t c1 = fork();
if (c1 == -1) {
    const char msg[] = "error: fork failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
if (c1 == 0) {
    if (out_fd != STDOUT_FILENO) close(out_fd);
    sem_close(to1); sem_close(from1);
    sem_close(to2); sem_close(from2);
    munmap(buf1, sizeof(shm_buffer_t));
    munmap(buf2, sizeof(shm_buffer_t));
    close(shm1_fd); close(shm2_fd);

    char *const args[] = {
        (char *)FILTER_NAME,
        shm1_name, sem_to1, sem_from1,
        NULL
    };
    execv(filterpath, args);

    const char msg[] = "error: exec failed\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
}
```

```
    pid_t c2 = fork();
    if (c2 == -1) {
        const char msg[] = "error: fork failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    if (c2 == 0) {
        if (out_fd != STDOUT_FILENO) close(out_fd);
        sem_close(to1); sem_close(from1);
        sem_close(to2); sem_close(from2);
        munmap(buf1, sizeof(shm_buffer_t));
        munmap(buf2, sizeof(shm_buffer_t));
        close(shm1_fd); close(shm2_fd);

        char *const args[] = {
            (char *)FILTER_NAME,
            shm2_name, sem_to2, sem_from2,
            NULL
        };
        execv(filterpath, args);

        const char msg[] = "error: exec failed\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    char line[BUF_SIZE];
    bool running = true;

    while (running) {
        ssize_t n = read_line(STDIN_FILENO, line, sizeof(line) - 1);
        if (n < 0) {
            const char msg[] = "error: read from stdin failed\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
        if (n == 0 || (n == 1 && line[0] == '\n')) {
            running = false;
            break;
        }

        bool to_first = roll_80();
        shm_buffer_t *buf = to_first ? buf1 : buf2;
        sem_t *to_sem     = to_first ? to1  : to2;
        sem_t *from_sem   = to_first ? from1: from2;

        buf->len = (size_t)n;
        memcpy(buf->data, line, n);

        sem_post(to_sem);

        sem_wait(from_sem);

        write_all(STDOUT_FILENO, buf->data, buf->len);
        if (out_fd != STDOUT_FILENO) write_all(out_fd, buf->data, buf->len);
    }
```

```
    buf1->len = 0; sem_post(to1);
    buf2->len = 0; sem_post(to2);

    sem_wait(from1);
    sem_wait(from2);

    waitpid(c1, NULL, 0);
    waitpid(c2, NULL, 0);

    slot_destroy(buf1, shm1_fd, to1, from1,
                 shm1_name, sem_to1, sem_from1);
    slot_destroy(buf2, shm2_fd, to2, from2,
                 shm2_name, sem_to2, sem_from2);

    if (out_fd != STDOUT_FILENO) close(out_fd);

    return 0;
}
```

mmap_reverse.c

```
#include <unistd.h>
#include <stdlib.h>
#include <stddef.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <semaphore.h>

#define BUF_SIZE 4096

typedef struct {
    size_t len;
    char   data[BUF_SIZE];
} shm_buffer_t;

static void reverse(char *s, size_t len) {
    for (size_t i = 0, j = len - 1; i < j; ++i, --j) {
        char tmp = s[i];
        s[i] = s[j];
        s[j] = tmp;
    }
}

int main(int argc, char **argv) {
    if (argc < 4) {
        const char msg[] = "usage: reverse <shm_name> <sem_to> <sem_from>\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return EXIT_FAILURE;
    }

    const char *shm_name     = argv[1];
    const char *sem_to_name  = argv[2];
    const char *sem_from_name= argv[3];

    int fd = shm_open(shm_name, O_RDWR, 0);
```

```
    if (fd == -1) {
        const char msg[] = "reverse: failed to open SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return EXIT_FAILURE;
    }

    shm_buffer_t *buf = mmap(NULL, sizeof(shm_buffer_t),
                             PROT_READ | PROT_WRITE,
                             MAP_SHARED, fd, 0);
    if (buf == MAP_FAILED) {
        const char msg[] = "reverse: failed to map SHM\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        return EXIT_FAILURE;
    }
    close(fd);

    sem_t *to   = sem_open(sem_to_name,   0);
    sem_t *from = sem_open(sem_from_name, 0);
    if (to == SEM_FAILED || from == SEM_FAILED) {
        const char msg[] = "reverse: failed to open semaphore\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        munmap(buf, sizeof(shm_buffer_t));
        return EXIT_FAILURE;
    }

    for (;;) {
        sem_wait(to);

        if (buf->len == 0) {
            sem_post(from);
            break;
        }
        size_t start = 0;
        for (size_t i = 0; i < buf->len; ++i) {
            if (buf->data[i] == '\n') {
                reverse(buf->data + start, i - start);
                start = i + 1;
            }
        }
        if (start < buf->len) {
            reverse(buf->data + start, buf->len - start);
        }
        sem_post(from);
    }
    sem_close(to);
    sem_close(from);
    munmap(buf, sizeof(shm_buffer_t));
    return 0;
}
```

Протокол работы программы

Входной файл in.txt:

```
Test

tseT

fgffjlfjgl

laboratory task 1

operation system
```

Вывод:

```
./mmap_client ./ mmap_reverse mmap_out.txt < mmap_in.txt

tseT

Test

lgjfljffgf

1 ksat yrotarobal

metsys noitarepo
```

Вывод

В ходе работы первая лабораторная работа была переведена с каналов (pipe) на именованную разделяемую память POSIX: вместо четырёх pipe создаются два объекта shm_open + ftruncate+ mmap (по одному на каждого дочернего процесса-фильтра), а данные передаются через memcpy в общий буфер, без копирования через ядро. Поскольку блокирующие read/write в канал больше не синхронизируют читателя и писателя, для каждого слота введены два именованных семафора (sem_open): to_child (начальное значение 0) будит ребёнка после записи данных, from_child (начальное значение 0) сообщает родителю о готовности результата; сигналом завершения служит len == 0 в заголовке буфера. Имена SHM и семафоров генерируются динамически от getpid(), что исключает коллизии между экземплярами программ, а корректный порядок sem_post/sem_wait и «квитанция» sem_post(from) перед выходом ребёнка гарантируют отсутствие гонок при чтении, записи и завершении. В результате работа демонстрирует разницу между созданием объекта разделяемой памяти (shm_open) и его отображением в адресное пространство (mmap), а также необходимость явной синхронизации при отказе от блокирующего I/O.